Claude Code Cheatsheet

Daily essentials for maximum productivity

v3.44.1 | October 2026

How to install

```
curl -fsSL https://claude.ai/install.sh | bash
# Verify installation
which claude && claude --version
```

Health Check: `claude doctor && claude mcp list`

Essential commands

Command	Action
`/help`	Contextual help
`/powerup`	Interactive animated lessons teaching Claude Code features
`/clear`	Reset conversation
`/compact`	Free up context
`/status`	Status tab: version, model, account, connectivity
`/context`	Detailed token breakdown
`/plan`	Enter Plan Mode (no changes)
Approve the plan	Exit Plan Mode (`Shift+Tab` to leave without approving)
`/model`	Choose model; Enter saves, s applies once
`/insights`	Usage analytics + optimization report
`/simplify`	Cleanup-only review of changed code: reuse, simplification, efficiency
`/batch`	Large-scale refactors via 5-30 parallel worktree agents
`/teleport`	Teleport session from web
`/tasks`	Monitor background tasks
`/remote-env`	Configure cloud environment
`/rc`	Start remote control session (Pro/Max/Team/Enterprise)
`/mobile`	Get Claude mobile app download links

Command	Action
`/fast`	Toggle fast mode (Opus 5.5: $8/$40 per MTok)
`/voice`	Toggle voice dictation (hold or tap Space)
`/recap`	One-line summary of the current session, on demand
`/effort [level]`	Thinking depth: low/medium/high/xhigh/max
`/loop [interval] [prompt]`	Run a prompt on repeat (self-paced when no interval is given); bundled skill
`/goal [condition]`	Autonomous multi-turn mode until condition is met
`/rename [name]`	Name or rename the current session
`/debug`	Enable debug logging and troubleshoot from the log (bundled skill)
`/exit`	Quit (alias `/quit`, or Ctrl+D)

Keyboard shortcuts

Shortcut	Action
`Shift+Tab`	Cycle permission modes
`Esc × 2`	Rewind menu (empty prompt) or clear draft
`Ctrl+C`	Interrupt (idle: clear input, press twice to exit)
`Ctrl+R`	Search command history
`Ctrl+L`	Redraw screen (keeps context)
`Ctrl+B`	Background tasks
`Ctrl+X Ctrl+K`	Kill all background agents
`Alt+T`	Toggle thinking where supported (not on Opus/Sonnet/Haiku 5.5, Fable)
`Space` (hold)	Voice input (requires `/voice`)
`Tab`	Accept autocomplete suggestion

Shortcut	Action
`Shift+Enter`	New line
`Ctrl+D`	Exit

IDE Shortcuts: VS Code `Alt+K` | JetBrains `Cmd+Option+K`

File references

```
@path/to/file.ts    → Reference a file
Ask for delegation → Claude may use an available agent
!shell-command      → Run shell command
```

Lesser-Known features (but official!)

Feature	What It Does
Tasks API	Persistent dependencies; newer models require `CLAUDE_CODE_ENABLE_TODO_TOOLS=1`
Background Agents	Sub-agents work while you code
Agent Teams	Multi-agent coordination (TeamCreate/SendMessage)
Auto-Memories	Automatic cross-session context capture
Session Forking	`/branch` branches the conversation; `/fork` copies it to a background session
LSP Tool	Definitions and references, according to configured servers
Voice Mode	Native voice input, free transcription
Remote Control	Control local session from phone/browser (Pro/Max/Team/Enterprise)
Scheduled Tasks	Cloud routines can run machine-off; Desktop tasks need the app and machine running
Skill Evals	A/B testing and trigger tuning for custom skills
Output Styles	`/output-style` : Default, Proactive, Concise, Explanatory, Learning
Rules Templates	Ready-to-use `.claude/rules/` templates (arch/quality/test/perf)

Pro tip: These aren't "secrets", they're in the CHANGELOG. Read it!

Permission modes

Mode	Editing	Execution
Manual (`default`)	Asks	Asks
acceptEdits	Auto	Asks
Plan Mode	None	Read-only tools
auto	Classifier decides	Classifier decides
dontAsk	Only if in allow rules	Only if in allow rules
Bypass	Auto	Auto (CI/CD only)

The Bypass mode identifier is `bypassPermissions` .

Shift+Tab to switch modes. Since v2.1.283 new sessions start in auto mode unless `permissions.defaultMode` or `--permission-mode` overrides it; `/auto-mode-setup` guides setup.

Memory & Settings (2 levels)

Level	Path	Scope	Git
Project	`.claude/`	Team	Yes
Personal	`~/.claude/`	You (all projects)	No

Settings priority: Managed policy > CLI > project-local > project > user. CLAUDE.md instructions combine rather than replacing one another.

.claude/ folder structure

```
.claude/
├── CLAUDE.md          # Shared project instructions (committed)
├── settings.json      # Hooks (committed)
├── settings.local.json # Permissions (not committed)
├── agents/            # Custom agents
├── hooks/             # Event scripts
├── rules/             # Auto-loaded rules
│   ├── architecture-review.md
│   ├── code-quality-review.md
│   ├── test-review.md
│   └── performance-review.md
└── skills/            # Slash commands + knowledge modules (unified)
```

Context management (CRITICAL)

Statusline: `Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%`

✅ 0-50%	⚠️ 50-70%	🔶 70-90%	🛑 90%+

Watch `/context`. Compact at a useful boundary; save decisions before clearing. Fixed percentages are workflow heuristics, not model limits.

Sign	Action
Short responses	`/compact`
Frequent forgetting	`/clear`
>70% context	`/compact`
Task complete	`/clear`

Context recovery commands

Command	Usage
`/compact`	Summarize and free context
`/clear`	Fresh start
`/rewind`	Restore a checkpoint; Bash and external effects remain
`claude -c`	Resume last session
`claude -r <id>`	Resume specific session

Plan mode & thinking

Feature	Activation	Usage
Plan Mode	`Shift+Tab × 2` or `/plan`	Explore without modifying
OpusPlan	`/model opusplan`	Opus for planning, Sonnet for execution

Thinking and effort

Opus 5.5, Sonnet 5.5, Haiku 5.5, and Fable 5.1 use adaptive thinking. Legacy Haiku 4.5 has **no effort parameter**.

Current adaptive models support `low`, `medium`, `high`, `xhigh`, and `max`. Opus 4.6 and Sonnet 4.6 support `max` but not `xhigh`.

`/effort` saves levels through `xhigh` per model. Use **s** for a session-only choice. `max` is session-only; `/effort auto` clears the active model's saved choice.

`Option+T` / `Alt+T` toggles thinking where supported. It has no effect on Opus 5.5, Sonnet 5.5, Haiku 5.5, or Fable, whose thinking cannot be turned off. Lower effort for bounded work instead.

`ultrathink` adds a reasoning instruction without changing API effort. `think hard` is ordinary prompt text.

Typical workflow

```
1. Start session      → claude
2. Check context      → /status
3. Plan Mode          → Shift+Tab × 2 (for complex tasks)
4. Describe task      → Clear, specific prompt
5. Review changes     → Always read the diff!
6. Accept/Reject      → Enter/Esc
7. Verify             → Run tests
8. Commit             → When task complete
9. /compact           → When context >70%
```

Quick prompting formula

```
WHAT: [Concrete deliverable]
WHERE: [File paths]
HOW: [Constraints, approach]
VERIFY: [Success criteria]
```

Example:

```
Add input validation to the login form.
WHERE: src/components/LoginForm.tsx
HOW: Use Zod schema, show inline errors
VERIFY: Empty email shows error, invalid format shows error
```

MCP servers

Server	Purpose
Serena	Indexing + session memory + symbol search
grepai	Semantic search + call graph analysis
Context7	Library documentation
Sequential	Structured reasoning
Playwright	Browser automation
Postgres	Database queries
doobidoo	Semantic memory + Knowledge Graph

Serena memory: `write_memory()` / `read_memory()` / `list_memories()`

Check status: `/mcp`

Search tools quick reference

Quick decision: exact text → `rg` | exact name → `rg` / Serena | concept → grepai | structure → ast-grep

Task	Tool
Find exact text	`rg "TODO"`
Find auth-related code	`grepai search "authentication"`
Who calls a function	`grepai trace callers "login"`

Task	Tool
Get file structure	`serena get_symbols_overview`

CLI flags quick reference

Flag	Usage
`-p "query"`	Non-interactive mode (CI/CD)
`-c` / `--continue`	Continue last session
`-r` / `--resume <id>`	Resume specific session
`--teleport`	Teleport session from web
`--model sonnet`	Change model
`--add-dir ../lib`	Allow access outside CWD
`--permission-mode plan`	Plan mode
`--worktree` / `-w`	Run in isolated git worktree
`--max-budget-usd 5.00`	Max API spend limit (print mode)
`--dangerously-skip-permissions`	Auto-accept (use carefully)
`--debug='mcp'`	Debug MCP servers
`--allowedTools "Edit,Read"`	Pre-approve tools; –tools restricts availability
`--debug`	Debug output

Anti-patterns

❌ **Don't**

- Vague prompts
- Accept without reading
- Ignore warnings
- Skip permissions
- Negative constraints only

✅ **Do**

- Specify file + line with @references
- Read every diff
- Use /compact at 70%
- Never in production
- Provide alternatives

Cost optimization

Current models

Standard API rates verified October 6, 2026. USD per million input/output tokens, excluding caching.

Model	Input / output	Context	Default effort
Haiku 5.5	$0.10 / $0.50 (up to 100K; $0.50 / $2.50 above)	1M	medium
Sonnet 5.5	$2 / $10	1M	medium
Opus 5.5	$4 / $20	1M	medium
Fable 5.1	$10 / $50	1M	high

Sonnet's published rate remains $2/$10 after the launch promotion. Opus 5.5 is the direct-service default; account policy can override it. Model choice does not establish a typical task cost.

Model	Use For	Cost
Haiku	Simple fixes, reviews	$
Sonnet	Most development	$$
Opus	Architecture, complex bugs	$$$
OpusPlan	Plan (Opus) + Execute (Sonnet)	$$

Quick decision tree

Four-Layer quick map

Layer	Selection question	Reference
Model	Which reasoning and cost fit the job?	Glossary
Runtime harness	Who runs the tool-observation-decision loop?	Agent Harness Map
Repository harness	Which instructions, verification, and state bound the repository?	Agent Harness Engineering
Orchestrator	Do several runtimes or sessions need coordination?	Agent Tools

Evaluate the exact model-harness pair for a coding task. Add an orchestrator when coordination is the identified need. A harness optimizer sits outside the four operating layers and changes candidate harnesses under a separate evaluation protocol.

Loop and graph are not extra substitutes. Loop names the repeated feedback and stopping rule. Graph names the executable nodes, branches, and checkpoints inside the chosen structure. For observed

mechanisms, `confirmed` = visible mechanism, `claimed` = source assertion, `unknown` = point not established yet. See also Loop & Graph Engineering.

```
Simple task      → Just ask Claude
Complex task     → Plan first; opt in to task tools if needed
Risky change     → Plan Mode first
Repeating task   → Create agent or command
Context full     → /compact or /clear
Need docs        → Use Context7 MCP
Deep analysis    → Use Opus (thinking on by default)
```

The Golden Rules

1. Always review diffs before accepting
2. Use /compact before context gets critical (>70%)
3. Be specific in requests (WHAT, WHERE, HOW, VERIFY)
4. Plan Mode first for complex/risky tasks
5. Create CLAUDE.md for every project
6. Commit frequently after each completed task
7. Know what's sent: prompts, files, MCP results → Anthropic

Common issues quick fix

Problem	Solution
"Command not found"	Check PATH and `claude doctor`; repair the chosen installation
Context too high (>70%)	`/compact` immediately
Slow responses	`/compact` or `/clear`
MCP not working	`claude mcp list`, check config
Permission denied	Check `settings.local.json`
Hook blocking	Check hook exit code, review logic

Health Check: `which claude && claude doctor && claude mcp list`

Author: Florian BRUNIAUX | Methode Aristote | Written with Claude

Version 3.44.1 | October 2026